

Data Simulation



Loading libraries

```
library(sf)
library(ggplot2)
library(spdep)
library(Matrix)
library(MASS)
library(dplyr)
library(sp)
```

In this workflow, we simulate spatial domain. Then, we define regions inside the domain. We then simulate data from CAR model for each region. Finally, we combine the data from all regions and save it.

Parameter settings

```
k_sim <- 8# spatil distance nearest neighbors
p <- 5
eta <- 0.3
eta_1 <- eta
eta_2 <- eta
tau_sq <- 1
tau_sq_1 <- tau_sq
tau_sq_2 <- tau_sq
mu_1 = 7
mu_2 = 10

# grid one axis
dim <- 40
```

```
# Number of points to simulate
nrw <- 50
ncl <- 50
n <- nrw * ncl
```

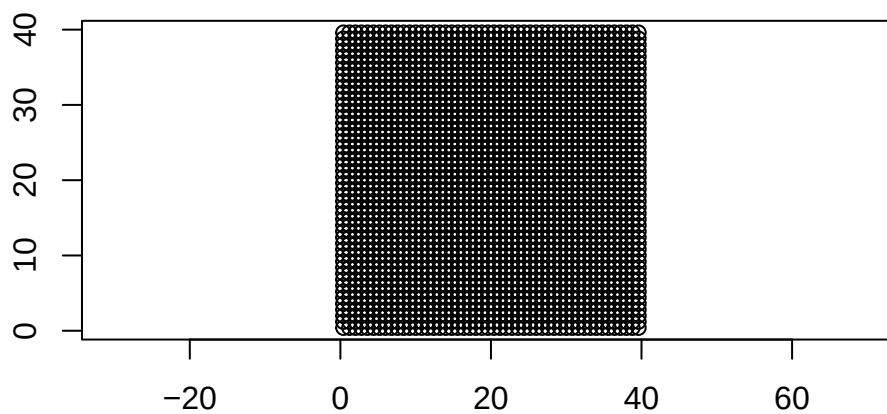
Simulate coordinates and regions

Generating 'n' coordinates inside dim x dim grid

```
# Defining the domain size as an sfc object
sfc <- st_sfc(
  st_polygon(
    list(rbind(c(0,0), c(dim,0), c(dim,dim), c(0,dim), c(0,0)))
  )
)

# creates regular grid of points over the over the sfc
coordinates <- st_make_grid(
  sfc,
  n=c(nrw,ncl),
  what = "centers"
)

plot(coordinates, axes = TRUE)
```



Define regions inside the sfc

```
# Defining circle 1
center1 <- c(33,8)
radius1 <- 5
circle1 <- st_buffer(st_sfc(st_point(center1)), dist = radius1)

# Defining circle 2
center2 <- c(8,33)
radius2 <- 5
circle2 <- st_buffer(st_sfc(st_point(center2)), dist = radius2)

# Defining quadrangle
length_quad <- 9
corner_coordinates_1 <- list(
  rbind(
    c(3,3),
    c(3+length_quad,3),
    c(3+length_quad,3+length_quad),
    c(3,3+length_quad), c(3,3))
)

square <- st_polygon(corner_coordinates_1)

# Defining triangle
corner_coordinates_2 <- list(rbind(c(28,31), c(38,31), c(33,38), c(28,31)))
triangle <- st_polygon(corner_coordinates_2)

# Defining center larger circle and smaller circle at the center
centers <- c(20,20)
radius_l <- 8
radius_s <- 5

large_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_l)
small_circle <- st_buffer(st_sfc(st_point(centers)), dist = radius_s)

# Create a ring by subtracting the smaller circle from the larger circle
ring <- st_difference(large_circle, small_circle)
```

Extract coordinates are for regions?

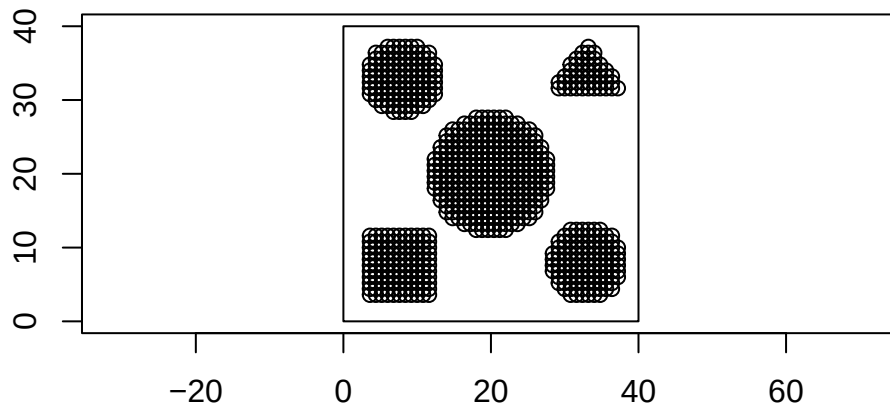
```
# Circle 1
circle_1_coords <- st_intersection(coordinates, circle1)
# Quadrangle
quad_coords <- st_intersection(coordinates, square)
# Circle 2
circle_2_coords <- st_intersection(coordinates, circle2)
# Triangle
tri_coords <- st_intersection(coordinates, triangle)
# Ring
ring_coords <- st_intersection(coordinates, ring)
# Circle inside the ring
circle_s_coords <- st_intersection(st_intersection(coordinates, small_circle))
```

number of coordinates inside each region

```
num_circle_1 <- length(circle_1_coords)
num_quad <- length(quad_coords)
num_circle_2 <- length(circle_2_coords)
num_tri <- length(tri_coords)
num_ring <- length(ring_coords)
num_circle_s <- length(circle_s_coords)
```

Plot all the captured regions

```
cap_coords <- st_sf(
  coords = c(
    circle_1_coords,
    circle_2_coords,
    quad_coords,
    tri_coords,
    circle_s_coords,
    ring_coords
  )
)
plot(sfc, axes = TRUE)
plot(cap_coords, add = TRUE)
```



Coordinates in the un-captured region (noise)

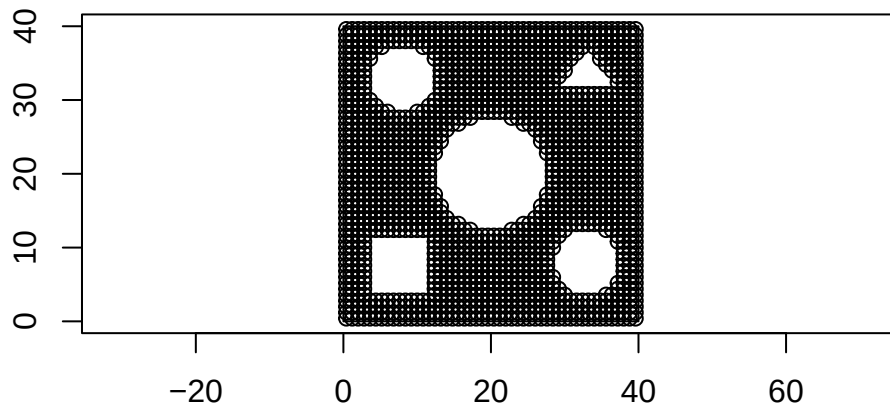
```
# Function to create an "sf object" of the union of the shapes
union_sf <- function(sf_objects) {
  result <- sf_objects[[1]]
  for (i in 2:length(sf_objects)) {
    result <- st_union(result, sf_objects[[i]])
  }
  return(result)
}

sf_list <- list(circle1, circle2, square, triangle, large_circle)

union_result <- union_sf(sf_list)

# Coordinates of the noise
non_coords <- st_sf(
  coords = st_difference(coordinates, union_result)
)

plot(sfc, axes = TRUE)
plot(non_coords, add = TRUE)
```



CAR model for each region

Function for simulations from spatial models

```
# Function to simulate for the selected regions
simulate_CAR <- function(
  n,
  coords,
  k,
  mu,
  tau_sq,
  eta){

  # Creating k-nearest neighbors
  nb_simulation <- knn2nb(
    knearneigh(
      coords,
      k
    )
  )

  # Making the neighborhood structure un-directional
  nb_1_sym <- make.sym.nb(
    nb_simulation
  )

  # Weight matrix for simulation
  w_mat <- nb2mat(
    nb_1_sym,
    style = "W"# row standardized
  )
}
```

```

    )

# Defining C matrix
c <- eta*w_mat

# Defining mean matrix
mu_mat <- rep(mu,n)

# Defining sigma matrix
# Defining sigma matrix
identity <- diag(
  seq(1,1,length=n)
)

sigma_mat <- tau_sq*solve(identity-c)

# Simulating from joint distribution of the variables from CAR model (MVN)
z_CAR <- mvrnorm(
  n=1, # one realization for the region (patch)
  mu_mat,
  sigma_mat
)

z <- as.vector(z_CAR)

return(z)
}

```

Noise simulation

- Function for simulating noise

```

simulate_noise <- function(
  n,
  coords,
  p, # number of attributes
  mu,
  sigma
){

```

```

# Initiating mean and sigma
mu_mat <- rep(mu,p)

sigma_mat <- diag(rep(sigma, p))

# Simulating data from the multivariate normal distribution
noise <- mvrnorm(
  n=n,
  mu=mu_mat,
  Sigma = sigma_mat
)

colnames(noise) <- paste0("Z",1:p)
data <- cbind(coords, noise)

return(data)
}

```

Parameters set 1

- Simulating data for circle1, quadrangle and ring from the same distribution (CAR model)

I want to show empirically that the MMD^2 is consistent (or the variance is finite) –Monte carlo method

So I need to simulate many (200) samples for the patches in region 1. Let's write a function to do this

The following function simulate data for region 1 with parameters set 1. We use the coordinates for the regions and number of coordinates in each region computed above. So they are global objects.

```

region1 <- function(
  k_sim,
  p,
  mu_1,
  tau_sq_1,
  eta_1
){

# Set of regions

```



```

regions_1 <- list(
  st_sf(coords = circle_1_coords),
  st_sf(coords = quad_coords),
  st_sf(coords = ring_coords)
)

# Number of coordinates for each region
regions_1_num <- c(
  num_circle_1,
  num_quad,
  num_ring
)

# an empty list to store the data
results_list_1 <- list()

for (i in 1:length(regions_1_num)) {
  # p variables
  sim_data_1 <- data.frame(
    lapply(1:p,
      function(x){
        simulate_CAR(
          regions_1_num[i],
          regions_1[[i]],
          k_sim,
          mu_1,
          tau_sq_1,
          eta_1
        )
      }
    ),
    coords = regions_1[[i]]
  )

  names(sim_data_1)[1:p] <- paste0("Z", 1:p)

  results_list_1[[i]] <- sim_data_1
}

return(results_list_1)

```

```
}
```

Parameter set 2

- Simulating data for triangle, circle 2 and inner ring circle

```
region2 <- function(  
  k_sim,  
  p,  
  mu_2,  
  tau_sq_2,  
  eta_2  
) {  
  # Set of regions  
  regions_2 <- list(  
    st_sf(coords = circle_2_coords),  
    st_sf(coords = tri_coords),  
    st_sf(coords = circle_s_coords)  
  )  
  
  # Number of coordinates for each region  
  regions_2_num <- c(  
    num_circle_2,  
    num_tri,  
    num_circle_s  
  )  
  
  # empty list to store the results  
  results_list_2 <- list()  
  
  for (i in 1:length(regions_2_num)) {  
    # i is the region (patch) in Region 2  
    sim_data_2 <- data.frame(  
      lapply(  
        1:p,  
        function(x) {  
          simulate_CAR(regions_2_num[i],  
            regions_2[[i]],  
            k_sim,  
            mu_2,  
            tau_sq_2,
```

```

        eta_2)
    }
  ),
  coords = regions_2[[i]]
)

names(sim_data_2)[1:p] <- paste0("Z", 1:p)

results_list_2[[i]] <- sim_data_2
}

return(results_list_2)
}

```

Combine data from two regions

- skip noise

```

# k_sim = 8# spatil distance nearest neighbors
# p = 5
# eta = 0.3
# eta_1 = eta
# eta_2 = eta
# tau_sq = 1
# tau_sq_1 = tau_sq
# tau_sq_2 = tau_sq
# mu_1 = 7
# mu_2 = 10

regionAll <- function(
  k_sim,
  p,
  mu_1,
  tau_sq_1,
  eta_1,
  mu_2,
  tau_sq_2,
  eta_2
){
  results_list_1 <- region1(

```

```

    k_sim = k_sim,
    p = p,
    mu_1 = mu_1,
    tau_sq_1 = tau_sq_1,
    eta_1 = eta_1
  )

results_list_2 <- region2(
  k_sim = k_sim,
  p = p,
  mu_2 = mu_2,
  tau_sq_2 = tau_sq_2,
  eta_2 = eta_2
)

# Combining the two lists of region 1 and region 2 data
results_list <- c(
  results_list_1,
  results_list_2
)

names(results_list) <- c(
  "circle1",
  "quad",
  "ring",
  "circle2",
  "tri",
  "circle_s"
)

# convert the list to a data frame with a column for the name of the region
data_regions <- results_list |>
  bind_rows(.id = "region")
# convert the dataframe to a sf data frame
data_regions <- st_as_sf(data_regions, sf_column_name = "coords")
return(data_regions)
}

# need to do this for 100 runs and save it

data_run <- regionAll(
  k_sim = k_sim,

```

```

    p = p,
    mu_1 = mu_1,
    tau_sq_1 = tau_sq_1,
    eta_1 = eta_1,
    mu_2 = mu_2,
    tau_sq_2 = tau_sq_2,
    eta_2 = eta_2
)

```

```

R <- 100# number of geospatial realizations (simulation runs for each setting)

#empty list to store results
results_list <- vector("list", R)

#run regionAll R times and store each result
for (r in 1:R) {
  results_list[[r]] <- regionAll(
    k_sim = k_sim,
    p = p,
    mu_1 = mu_1,
    tau_sq_1 = tau_sq_1,
    eta_1 = eta_1,
    mu_2 = mu_2,
    tau_sq_2 = tau_sq_2,
    eta_2 = eta_2
  )
}

```

```

#save the list to an RData file
filen <- paste0("sim_eta_",eta,"_p_",p,"_n_",n,".RData")
save(results_list, file = filen)

# filen <- paste0("sim_eta1_",eta_1,"_eta2_",eta_2,"_p_",p,"_n_",n,".RData")
# save(results_list, file = filen)

```